

Exercise for MA-INF 2213 Computer Vision SS26

20.05.2026

Submission deadline: 10.06.2026

Important: We recommend using **Python 3.9** at least. We recommend **PyTorch (version 2.3)** and **torchvision (version 0.18)**.

Grading: Submissions that generate runtime errors or produce obviously rubbish results (e.g., nans, inf) will receive at most 50% of the points.

Plagiarism: Plagiarism in any form is prohibited. If your solution contains code copied from any source (e.g. other students, or solutions from the web), you will receive **0 points** for the entire exercise sheet.

Submission: You can complete the exercise in groups of two, but only one submission per group is allowed. Include a *README.txt* file with your group members. Points for solutions without a README will only be given to the uploader.

Fine-Grained Visual Classification on CUB-200-2011

The objective of this assignment is to develop and systematically improve a convolutional neural network for **fine-grained bird species classification** using a **25-class subset** of the CUB-200-2011 dataset.

Note: This task and dataset are intentionally challenging. Fine-grained classification on a small training set is a hard problem, so do not expect high accuracy. Focus instead on understanding the effect of each modification and providing a thorough discussion in your report.

You are allowed to use **only** methods and architectural components explicitly covered in the lecture notes (Neural Networks, MA-INF 2213, SS26).

Usage of any component **not** covered in the lecture is strictly prohibited. The motivation behind every architectural choice must be clearly explained in the report.

Dataset

We use a **25-class subset** of the CUB-200-2011 dataset (Caltech-UCSD Birds-200-2011), which contains high-resolution photographs of bird species with fine-grained inter-class differences.

Data split:

- You will be provided with **80% of the data** (training split).
- You must further split this into a **training set** and a **validation set** (we recommend an 80/20 ratio within the provided split, but you may choose differently and should justify your choice in the report).
- The remaining **20% (test set)** is held out by the instructors and will be used to evaluate your final model after submission.
- Do **not** include the dataset in your submission.

Note on image resolution: CUB-200-2011 images are higher resolution than CIFAR-10. Resize all images to 128×128 (or 224×224 if memory permits) before feeding them into the network. Describe your pre-processing choices in the report.

Regarding the code

A skeleton notebook is provided. It includes the dataset class, data loaders, training loop, evaluation loop, and the baseline architecture. The random seed is fixed — do not change it.

Tasks 1–4: The `__init__` method of each class is already filled in. You only need to implement the `forward` method where indicated by `# YOUR CODE HERE`.

Task 5: Implement both `__init__` and `forward` from scratch.

Submission

Technical report (maximum of 3 pages):

The report must briefly describe the implementation and provide a thorough discussion of results. Cover the following:

- **Introduction:** The task, dataset (25 classes, 128×128 , train/val split), and base architecture.
- **Experiments:** Accuracy, training curves, and parameter count for each task separately, using tables and figures.
- **Discussion (can be merged with experiments):** The report is expected to have a thorough analysis and a detailed discussion about the results obtained for each question.
- **Conclusion:** Summary of findings and most impactful improvements.

Completed code:

Submit the completed notebook with all `forward` methods implemented for Tasks 1–4. For Task 5, place the following in a `task5/` subfolder:

- **model.py** — Self-contained file defining your `YourNet` class. Must be importable as `from model import YourNet` without any other files.
- **best_model.pth** — Weights saved with `torch.save(model.state_dict(), ...)`.

The completed notebook must also be included. If you use additional libraries, include a `requirements.txt`.

Notes

- The maximum allowed number of parameters for your final model (Task 5) is **500,000**.

- The maximum number of training epochs is **50**.
- Report the best accuracy across all 50 epochs and the exact epoch number.
- Do not include the dataset in your submission.
- Do not use pre-trained weights or external model implementations.

Grading

You can earn **20 points** from this exercise sheet.

Task 1: Pooling Strategies *(3 points)*

Modify the base network by (a) replacing MaxPool with AvgPool, and (b) using strided convolutions instead of pooling. Report accuracy and training time for both variants against the baseline.

Task 2: Activation Function Comparison *(2 points)*

Replace all ReLU activations with (a) Leaky ReLU and (b) GELU. Compare and discuss results against the baseline.

Task 3: Normalization Layers *(3 points)*

Starting from a BN-free network, add (a) Batch Normalization and (b) Group Normalization. Compare both against the no-norm reference. You may tune batch size and learning rate.

Task 4: Depthwise Separable Convolutions *(2 points)*

Replace all convolutions with depthwise separable convolution blocks. Compare accuracy, parameter count, and inference time against the baseline.

Task 5: Best Architecture *(5 points)*

Design and implement YourNet from scratch. Maximum **500,000 parameters** and **50 epochs**. Submit `task5/model.py` and `task5/best_model.pth`.

Technical report (maximum of 3 pages) *(5 points)*

The technical report must briefly describe the implementation and provide a detailed discussion of the results for each task.

Bonus: The three submissions achieving the best test-set accuracy will receive **2 bonus points**.